18.3 合并（Coalescing）与人性化节奏

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~500,000 tokens，输出 ~40,000 tokens（本章合计）

块流解决了"把连续文本切成合适大小的块"的问题，但还有一个体验问题——如果 Agent 快速生成了多个短块，用户会收到一连串"叮叮叮"的消息通知。**合并（Coalescing）** 机制将短时间内产生的多个块合并为一条消息；**人性化延迟（Human Delay）** 则模拟人类的打字节奏，让 Agent 的回复不那么"机器感"。

18.3.1 连续块合并策略：空闲间隙、最小/最大字符数

块发射控制

在 `subscribeEmbeddedPiSession` 中， `emitBlockChunk` 是块发射的统一出口。它在发射前执行多重过滤：

```
// src/agents/pi-embedded-subscribe.ts（简化）
const emitBlockChunk = (text: string) => {
  if (state.suppressBlockChunks) return;     // 压制中（已有最终文本）

  // 1. 剥离 <think>/<final> 标签
  const chunk = stripBlockTags(text, state.blockState).trimEnd();
  if (!chunk) return;

  // 2. 去重：与上一次发射的相同 → 跳过
  if (chunk === state.lastBlockReplyText) return;

  // 3. 消息工具去重：已通过 messaging tool 发送 → 跳过
  const normalizedChunk = normalizeTextForComparison(chunk);
  if (isMessagingToolDuplicateNormalized(
    normalizedChunk, messagingToolSentTextsNormalized
  )) return;

  // 4. 助手文本去重：与最近的 assistantText 相同 → 跳过
  if (shouldSkipAssistantText(chunk)) return;

  // 5. 记录并发射
  state.lastBlockReplyText = chunk;
  assistantTexts.push(chunk);

  if (!params.onBlockReply) return;

  // 6. 解析回复指令（媒体 URL、音频标记、引用标签）
  const splitResult = replyDirectiveAccumulator.consume(chunk);
  if (!splitResult) return;

  void params.onBlockReply({
    text: splitResult.text,
    mediaUrls: splitResult.mediaUrls,
    audioAsVoice: splitResult.audioAsVoice,
    replyToId: splitResult.replyToId,
    replyToCurrent: splitResult.replyToCurrent,
  });
};
```

块边界模式

块的边界由 `blockReplyBreak` 参数控制：

模式	行为	适用场景
`"text_end"`	每个 `text_end` 事件触发块刷新	实时流（Telegram 草稿模式）
`"message_end"`	仅在 `message_end` 时刷新	完整消息（电子邮件、短信）

在 `text_end` 模式下，每次收到文本结束事件时都会强制刷新块分块器：

```
// src/agents/pi-embedded-subscribe.handlers.messages.ts
if (evtType === "text_end" && state.blockReplyBreak === "text_end") {
  if (ctx.blockChunker?.hasBuffered()) {
    ctx.blockChunker.drain({ force: true, emit: ctx.emitBlockChunk });
    ctx.blockChunker.reset();
  } else if (ctx.state.blockBuffer.length > 0) {
    ctx.emitBlockChunk(ctx.state.blockBuffer);
    ctx.state.blockBuffer = "";
  }
}
```

工具执行前的强制刷新

当工具开始执行时，需要先刷新已有的文本块——确保用户在看到工具操作之前，已经看到了 Agent 的所有文字解释：

```
// src/agents/pi-embedded-subscribe.handlers.tools.ts
export async function handleToolExecutionStart(ctx, evt) {
  // 刷新待发送的文本块
  ctx.flushBlockReplyBuffer();
  if (ctx.params.onBlockReplyFlush) {
    void ctx.params.onBlockReplyFlush();
  }
  // ...后续工具处理
}
```

消息边界重置

每个新的 assistant 消息开始时，所有块状态会被重置：

```
// message_start 事件触发
const resetAssistantMessageState = (nextAssistantTextBaseline: number) => {
  state.deltaBuffer = "";
  state.blockBuffer = "";
  blockChunker?.reset();
  state.blockState.thinking = false;
  state.blockState.final = false;
  state.lastBlockReplyText = undefined;
  state.suppressBlockChunks = false;
  state.assistantMessageIndex += 1;
  state.assistantTextBaseline = nextAssistantTextBaseline;
};
```

18.3.2 人性化延迟（Human Delay）： `natural` / `custom` 模式

在 18.1 节里我们提到了 `resolveHumanDelayConfig` ，它返回人性化延迟的配置。在消息发送层面，这个延迟会被应用到块回复的发射间隔上。

配置

```
agents:
  defaults:
    humanDelay:
      mode: "typing"     # 显示"正在输入"状态
      minMs: 500          # 最短延迟
      maxMs: 3000         # 最长延迟
```

工作原理

人性化延迟并不在 `subscribeEmbeddedPiSession` 内部实现——它在**消息发送层**（如 Telegram bot 适配器、Signal 适配器）中应用。当 `onBlockReply` 回调被触发时，适配器会：

1. 发送"正在输入"（typing）状态指示
2. 等待一个随机的延迟时间（ `minMs` ~ `maxMs` ）
3. 发送实际的消息内容

这种设计把内部的流式传输逻辑（ `subscribeEmbeddedPiSession` ）和外部的通道适配器分开了——内部总是尽快处理，延迟由外层按需添加。

压缩期间的缓冲重置

当自动压缩触发时，所有已缓冲的块和状态需要被清除：

```
const resetForCompactionRetry = () => {
  assistantTexts.length = 0;     // 清空助手文本
  toolMetas.length = 0;          // 清空工具元数据
  toolMetaById.clear();
  state.lastToolError = undefined;
  messagingToolSentTexts.length = 0;  // 清空消息去重记录
  resetAssistantMessageState(0);        // 重置块状态
};
```

这确保压缩后的重试从完全干净的状态开始——用户不会看到压缩前的部分文本紧接着压缩后的新文本。

本节小结

1. **块发射**（`emitBlockChunk`）在发射前执行五重过滤：压制检查、标签剥离、文本去重、消息工具去重、助手文本去重。
2. **两种边界模式**（`text_end` / `message_end`）适应不同通道的需求——实时通道倾向频繁刷新，批量通道倾向完整消息。
3. **工具执行前刷新**确保用户在看到工具操作前已收到所有解释文本。
4. **人性化延迟**在消息发送层（通道适配器）实现，与流式传输内部解耦。
5. **压缩重置**确保自动压缩后，流式传输从干净状态重新开始。